

Architectural Blueprint and Implementation Strategy for the Canary Multi-User Smart Assistant

THINGS TO IMPLEMENT →

1. Use cases to address
2. Efficient pre processing , silent VAD, then forwarding the queries
3. Removing Background noise
4. Dynamic Scaling of resources, can use something Agentic/ With AI/ ML if enough parameters/ time is left over
5. Scaling of parameters has to be clearly visible for the final demo (main idea)
6. Role Based Access Control
7. Agent Orchestration
8. Model training

Executive Overview

The transition from cloud-reliant, monolithic sequence transduction models toward highly localized, edge-based architectures represents a profound shift in computational acoustics. Developing an autonomous, real-time multi-user smart assistant—code-named "Canary"—requires successfully navigating dynamic, chaotic acoustic environments saturated with simultaneous human-to-human conversations, ambient noise, and overlapping device-directed commands.¹ **The primary engineering challenge lies in adhering to strict operational constraints: the core multi-speaker separation model must not exceed 5.0 million parameters, the system must maintain a Real-Time Factor (xRT) of less than 0.5, and it must achieve a Scale-Invariant Signal-to-Noise Ratio (SI-SNR) greater than 25 dB for clean audio and greater than 10 dB in highly dense overlaps.**¹

Furthermore, a rigid development context dictates the entire architectural approach. The execution environment is defined by a two-person engineering team, a compressed two-to-three-week timeline, an absence of paid cloud GPU instances, and a strict open-source mandate.¹ These limitations categorically preclude the from-scratch training of massive joint acoustic models. The strategy must instead be rooted in the intelligent aggregation, quantization, and orchestration of pre-trained, open-weight neural topologies operating sequentially.¹

The exhaustive technical analysis presented herein provides a complete, mathematically grounded implementation strategy for the Canary project. It decisively rejects the paradigm of joint, end-to-end separation and Automatic Speech Recognition (ASR) modeling, proving that such architectures mathematically cannot operate within the 5.0 million parameter limit without suffering catastrophic mode collapse and high Word Error Rates (WER).¹ Instead, the strategy advocates for a **strictly decoupled Sequential Gated Modular Pipeline**. By orchestrating ultra-lightweight edge models—specifically Silero for Voice Activity Detection (VAD), microWakeWord for trigger detection, MatchboxNet for intent classification, the Time-Frequency Interleaved Gain Extraction and Reconstruction (TIGER) network for source separation, CAM++ for biometric verification, and Sherpa-ONNX executing SenseVoiceSmall for transcription—the pipeline dynamically allocates computational resources only to verified, device-directed speech.¹

The architecture bridges localized acoustic processing to an agentic reasoning engine. The localized lexical outputs are routed through a **Model Context Protocol (MCP) server**, effectively transforming the Canary system from a passive transcription engine into an autonomous orchestrator capable of resolving complex hierarchical conflicts, overlapping dialogue, and multi-step tool-chaining tasks natively on edge hardware.¹

Feasibility Analysis

Operating under the severe constraints of a two-person team over a maximum of twenty-one days without access to paid GPU clusters demands a brutally realistic assessment of what can be built, trained, and deployed. **Deep-tech acoustic challenges frequently require thousands of hours of parallelized GPU computation to train separation models on corpora like LibriMix or EchoSet.**¹ Because training an advanced time-frequency domain model from random initialization is computationally unfeasible within the project scope, the architecture must rely exclusively on pre-trained, open-weight models available via repositories such as Hugging Face and ModelScope.¹

The feasibility of the Canary project is mathematically guaranteed by the strategic interpretation of the parameter constraints. The foundational requirement states that the working model of the multi-speaker separation system must contain fewer than 5.0 million parameters.¹ The critical distinction is that this constraint applies exclusively to the core multi-speaker separation model and its directly associated front-end acoustic processing gates, rather than the entire end-to-end sequence transduction pipeline.¹ **Foundational ASR models mathematically cannot operate under this limit while maintaining an open vocabulary.**¹ **By deliberately decoupling the front-end signal processing from the backend ASR, the system**

successfully isolates the cocktail party problem from heavy linguistic decoding, preserving the 5.0M parameter budget solely for acoustic enhancement.¹

To categorize the realistic achievability of the system components, the engineering risks have been evaluated and classified.

Risk Category	Component Evaluation	Risk Level	Mitigation Strategy
Model Training	Training time-domain or frequency-domain separation models from scratch without paid GPU clusters.	HIGH	Abandon ground-up training entirely. <u>Rely exclusively on pre-trained Safetensor and ONNX weights</u> provided by researchers, leveraging zero-shot generalizability.
Latency / xRT	Running multiple neural networks (VAD, Wake-Word, Separation, ASR, LLM) on consumer-grade CPUs within a 0.5 xRT.	HIGH	Enforce aggressive INT8 quantization across all models. Utilize the ONNX Runtime for optimized C++ execution of Python inference graphs.
Integration	Bridging disparate audio streams from the separation module into parallel ASR instances and resolving asynchronous text outputs.	MEDIUM	Implement robust multi-threading and utilize continuous ring buffers in Python to prevent Global Interpreter Lock (GIL) freezing.
Demo Quality	Demonstrating complex acoustic separation live in a	MEDIUM	Pre-record specific overlapping audio datasets as a

	noisy hackathon hall with varying microphone hardware.		fallback. Implement a local caching system to demonstrate the software logic if live acoustics fail.
Deployment	Setting up a fully local agentic system without relying on cloud microservices.	LOW	Utilize containerized deployments (Docker) or self-contained Python environments (uv/virtualenv) and local SLM runners like Ollama.

The most significant risk is computational latency. Without cloud GPUs, the inference pipeline must be optimized for execution on local hardware, such as consumer-grade x86_64 architectures or ARM Cortex processors.¹ To achieve the mandated sub-0.5 xRT, the implementation strategy mandates the use of the Open Neural Network Exchange (ONNX) Runtime. ONNX Runtime provides extensive cross-platform compatibility and allows models trained in PyTorch or TensorFlow to be executed with extreme efficiency using INT8 quantization, drastically reducing memory bandwidth saturation on the host device.⁹

Minimum Viable Product (MVP) Definition

For a hackathon environment, the Minimum Viable Product (MVP) must prioritize impressive, highly visible functionalities that directly address the core problem statement, while ruthlessly cutting peripheral features that consume engineering time without adding demonstrative value. The MVP for Canary is defined by its ability to resolve the most challenging acoustic and semantic overlaps locally.

The smallest impressive working MVP requires the **system to ingest audio containing two simultaneous speakers, separate the voices with high fidelity, transcribe the speech, and utilize an agentic framework to resolve the overlapping intent.**

To achieve this within three weeks, specific elements must be cut or mocked:

- **Hardware Actuation:** The MVP will not integrate with actual physical IoT devices (smart bulbs, thermostats). Instead, the system will utilize mocked terminal outputs or graphical interface state changes to demonstrate that a command was routed successfully.
- **Dynamic Database Generation:** The system will not feature a front-end enrollment process for biometric voice mapping. **The speaker verification embeddings will be**

pre-calculated and hardcoded for the two team members prior to the demo.²

- **Unlimited Speaker Handling:** While models can technically handle three or more speakers, the MVP will restrict the separation logic to two overlapping speakers to guarantee processing speed and avoid permutation ambiguities during live inference.¹

The MVP will focus strictly on the following successful resolution pathways to maximize demo impact:

1. **Conflicting Directives:** If User 1 says "Turn on lights" and User 2 says "Turn off lights" simultaneously, the system must transcribe both, identify the conflict, and invoke the agent to request clarification.¹
2. **Hierarchical Privilege:** If a recognized "Admin" user and a "Guest" user issue conflicting commands, the agent must seamlessly execute the Admin's command based on pre-configured Role-Based Access Control (RBAC).¹
3. **Chatter Rejection:** The system must demonstrate its ability to ignore a background conversation that inadvertently contains the wake-word, proving the efficacy of its Acoustic Intent Gating.¹

Prioritizing these specific workflows ensures that the demo highlights the system's unique acoustic intelligence rather than generic language model capabilities.

System Architecture

To process overlapping speech efficiently without saturating edge hardware, the Canary system implements an acoustic intelligence funnel known as the Sequential Gated Modular Pipeline.¹ This architecture ensures that continuous audio events only trigger computationally expensive neural matrices when genuine communicative intent is cryptographically verified.¹ The architecture is divided into a frontend acoustic processing layer and a backend agentic reasoning layer, connected via synchronous data streams.

Audio Ingestion and Stage 0: Passive Idle Gate

The zero-layer of the architecture represents the perpetual listening state of the smart environment. Audio is captured via a standard microphone interface using libraries such as sounddevice or pyaudio, configured to ingest raw monaural audio at a 16 kHz sample rate.¹¹ The stream is buffered into continuous 30-millisecond chunks.

The primary gate is the Voice Activity Detection (VAD) model. If the VAD classifies the audio as silence, the buffer is dropped, and the system remains in a dormant state, minimizing thermal output.¹ Upon the detection of human speech energy, the audio buffer expands and is passed to the wake-word engine. If the wake-word model consistently predicts the target trigger over multiple sequential sliding windows, the system clears the Stage 0 threshold.¹²

Stage 1: Acoustic Scene Intelligence

Once awakened, the system must evaluate the context of the speech. A dedicated intent classification network evaluates the paralinguistic topography of the utterance.¹ The model simultaneously outputs a background noise rejection score, an overlap probability estimation, and a binary intent classification (Device-Directed vs. Ambient).¹ If the speech is classified as

ambient human-to-human conversation, the audio is dropped immediately. Following intent verification, the system calculates a Scene Complexity Score (SCS) based on the weighted sum of the overlap probability, the normalized noise floor, and the wake-word certainty.¹ This score dynamically routes the audio: clean, single-speaker audio bypasses heavy separation (Minimal Path), while heavily overlapping audio triggers the full multi-speaker separation stack (High-Complexity Path).¹

Stage 2: Multi-Speaker Separation and Biometrics

When the High-Complexity Path is triggered, the isolated 16 kHz audio buffer is passed to the core multi-speaker separation engine.¹ The separation network isolates the target speakers' waveforms, stripping away environmental noise and cross-talk.¹ In parallel, the separated target waveforms are routed to a speaker verification module. This module extracts unique biometric vocal embeddings and compares them via cosine similarity against a locally encrypted vector database (e.g., using FAISS) of known household users.¹ The verified identity is tagged to the separated audio channel as metadata.

Stage 3: Localized Sequence Transduction

The pristine, separated, and tagged single-speaker waveforms are subsequently fed into localized Automatic Speech Recognition (ASR) modules.¹ Because the streams are separated, the architecture instantiates parallel decoding threads to handle simultaneous speech without cross-pollination. The ASR engines convert the acoustic features into discrete text tokens, producing structured JSON outputs containing the transcription, the speaker ID, and precise timestamps.¹³

Stage 4: Agentic Orchestration

The resulting JSON arrays are ingested by a Python-based reasoning engine utilizing the Model Context Protocol (MCP).¹ The MCP server acts as a standardized bridge between a local Small Language Model (SLM) and external execution tools.⁴ The SLM evaluates the incoming semantic vectors, references the confidence scores calculated during earlier stages, and queries local databases for user permissions and routines.¹ The agent resolves overlapping logic, dictates task priorities, and executes the final Python functions to simulate hardware actuation.¹

Model and AI Research Integration

The success of the architecture relies entirely on selecting the most practical, highly compressed pre-trained models for each stage of the pipeline. The explicit requirement to utilize local models and eschew paid APIs necessitates an exhaustive evaluation of available open-weight topologies. The research definitively concludes that **fine-tuning should be avoided entirely in favor of leveraging the zero-shot generalization of highly optimized edge models.**

Wake-Word and VAD Topologies

For the initial energy gate, the system must deploy the Silero VAD model. Operating via an ONNX INT8 quantized backend, Silero requires approximately 0.50 million parameters and executes in less than a millisecond per chunk, making it the industry standard for lightweight edge detection.¹

To preserve the parameter budget for the separation task, the architecture eschews heavy embedding-based models for wake-word detection. The optimal choice is microWakeWord, developed by the Open Home Foundation.¹ This framework utilizes a TensorFlow Lite backend and operates on MixConv depthwise convolutions, demanding roughly 0.20 million parameters.¹ It processes 40 spectrogram features every 10 milliseconds, providing robust trigger detection with minimal false positives.¹²

Acoustic Intent Classification

In dynamic environments, wake-word homophony—where a trigger word aligns with casual conversational nouns—presents a severe vulnerability.¹ To construct an active Device-Directed Speech Detection (DDSD) layer at the edge, the system requires a network that analyzes paralinguistic cues without relying on expensive lexical transcription.¹

The recommended architecture is a $3 \times 2 \times 1$ variant of MatchboxNet.¹ MatchboxNet achieves unparalleled efficiency through its reliance on 1D time-channel separable convolutions, severing the spatial and channel convolution steps to drastically reduce computational costs.¹ By utilizing depthwise convolutions across frequency bands and pointwise convolutions to mix channels, **MatchboxNet captures the elongated phoneme durations and flattened pitch trajectories characteristic of machine-directed speech.**¹ Adapted for binary classification (Device-Directed vs. Ambient), this unified backbone consumes precisely 0.093 million parameters.¹

Multi-Speaker Separation Engines

The separation module is the critical bottleneck of the architecture. While models like the Convolutional Time-domain Audio Separation Network (Conv-TasNet) established baselines, their 5.1 million parameter count technically breaches the hackathon constraint.¹ Alternative architectures like SandglassNet (2.3 million parameters) utilize multi-granularity self-attention to capture global context and local features, achieving 17.1 dB SI-SNRi.¹ The Successive Downsampling and Resampling of Multi-Resolution Features (SuDORM-RF) architecture offers aggressive compression via U-Net style sub-blocks, with its 1.0x variant achieving 14.8 dB SI-SDRi at 2.6 million parameters.¹

However, the definitive SOTA recommendation for ultra-lightweight deployment is the Time-Frequency Interleaved Gain Extraction and Reconstruction (TIGER) network.¹ TIGER operates at an astonishing 0.8 million parameters.¹ **TIGER addresses the limitations of pure time-domain models by interleaving time and frequency domain processing.**¹ It utilizes a sophisticated frequency band-splitting strategy to compress spectral information, drastically reducing memory bandwidth.¹ Contextual features are efficiently extracted via a Multi-Scale

Selective Attention (MSA) module, while a Full-Frequency-Frame Attention module ensures global spectral continuity is maintained across the split bands.⁵

Crucially, the JuserLee/TIGER-speech repository provides fully accessible, pre-trained safetensor weights that have been rigorously evaluated.⁵ By implementing TIGER, the separation model reduces Multiply-Accumulate operations (MACs) by 95.3% compared to massive models like TF-GridNet, while achieving near-SOTA SI-SNR metrics well within the sub-0.5 xRT constraint.⁵

Biometric Speaker Verification

To fulfill the requirement of localized acoustic personalization, the architecture must extract biometric embeddings.¹ Heavily pruned models like ECAPA-TDNN frequently suffer from degraded Equal Error Rates (EER) in noisy conditions.¹ Because TIGER only consumes 0.8 million parameters, the system possesses the budget to deploy the Context-Aware Masking (CAM++) architecture, specifically the B0 configuration.¹

CAM++ leverages a densely connected time-delay neural network fused with multi-granularity pooling layers to capture global contextual dependencies across highly variable conversational lengths.¹ Meticulously engineered to operate at exactly **1.1 million parameters**, the pre-trained lovemefan/campplus ONNX runtime implementation executes without heavy PyTorch dependencies, occupying only 28 MB of storage.¹ This allows rapid inference and embedding extraction prior to downstream sequence transduction.

Localized Sequence Transduction (ASR)

The strict open-source mandate prohibits the transmission of audio to commercial cloud APIs.¹ The sequence transduction phase must be executed entirely **on-device**.¹ The optimal deployment strategy utilizes the k2-fsa/sherpa-onnx framework.⁸ Sherpa-ONNX is an open-source C++ toolkit engineered specifically for deploying ASR models locally, providing highly efficient Python bindings.⁸

Within this framework, the **SenseVoiceSmall model, developed by Alibaba's FunAudioLLM**, provides unparalleled edge performance.² Unlike traditional autoregressive architectures (like Whisper) that generate tokens sequentially, SenseVoiceSmall utilizes a non-autoregressive end-to-end framework.¹⁹ This allows it to calculate the entire transcription output in parallel, requiring only 70 milliseconds to process 10 seconds of audio.² Benchmarks demonstrate it operates at approximately 17 times real-time on standard CPUs, completely circumventing the need for GPU acceleration.¹³ Furthermore, SenseVoice is heavily optimized for multilingual recognition and code-switching, supporting over 50 languages natively.²

Agentic Frameworks and Small Language Models

To enable complex reasoning, the system must deploy **agentic workflows**.¹ The architecture relies on the Model Context Protocol (MCP), specifically integrating the FastMCP Python framework.¹⁴ The MCP server provides a standardized interface allowing a local Small

Language Model (SLM) to securely execute Python functions as "tools".¹⁴

For the reasoning engine, the system will utilize an **SLM such as Qwen2.5-1.5B-Instruct or Llama-3.2-1B, hosted locally via the Ollama framework**. These highly quantized (e.g., 4-bit) models possess robust instruction-following capabilities. By feeding the SLM the structured JSON outputs from the acoustic pipeline—including transcriptions, speaker IDs, and confidence scores—the agent can autonomously decide whether to execute a command, resolve a privilege conflict, or ask the user for clarification.¹

Parameter Constraint Verification

To definitively prove feasibility, the parameter budget for the acoustic front-end must be audited against the 5.0 million parameter limit.¹

Functional Pipeline Stage	Optimized Neural Topology	Parameter Footprint	Justification
Passive Energy Gating	Silero VAD (ONNX INT8)	~0.50 M	Continuous asynchronous noise gating.
Wake Word Detection	microWakeWord	~0.20 M	Ultra-lightweight MixConv architecture.
Intent & Noise Gating	MatchboxNet ($3 \times 2 \times 1$)	0.093 M	Multi-class intent and noise rejection.
Speaker Separation	TIGER (Pre-trained)	~0.80 M	Primary SOTA cocktail-party resolution.
Speaker Verification	CAM++ (B0 Variant ONNX)	~1.10 M	Superior EER without aggressive pruning.
Arbitration Engine	Python Rule-Based Logic	0.00 M	Deterministic weighted scoring.
Total Core Frontend	Canary MVP Pipeline	~2.693 M	Strictly < 5.0 M

By deploying TIGER and CAM++, the total acoustic frontend consumes merely 2.693 million parameters.¹ This provides a massive 2.3 million parameter safety buffer, ensuring absolute compliance with the challenge constraints while guaranteeing high-fidelity signal processing.

Data Requirements and Evaluation Strategies

Because the implementation strategy relies entirely on integrating pre-trained open-weight models, the project requires zero data engineering for backpropagation or ground-up training. Datasets are utilized exclusively for offline validation, hyperparameter tuning, and contextual prompting.

Acoustic Validation Datasets: The separation module (TIGER) must be evaluated against the LibriMix and EchoSet datasets.¹ LibriMix provides standard multi-speaker overlaps derived from the LibriSpeech corpus, establishing baseline SI-SNR metrics.¹ EchoSet provides crucial high-fidelity validation regarding object occlusions and real-world room impulse responses (RIR), ensuring the model will survive a noisy hackathon hall.⁵

Transcription and Logic Datasets: The ASR pipeline will be validated using samples from the IndicVoices dataset to ensure the SenseVoice model effectively resolves regional accents and code-switching.¹ Furthermore, to optimize the arbitration engine's logic for smart home operations, the developers will utilize the SmartSense dataset.¹ SmartSense is a repository of millions of IoT logs and routines collected via the SmartThings platform.¹⁵ By extracting common routine dictionaries from this dataset, the engineering team can inject realistic smart-home contextual priors into the SLM's system prompt, allowing the agent to make highly accurate deductive leaps during conflict resolution.¹

Implementation Roadmap

Successfully architecting this deeply integrated system within a rigid twenty-one-day timeframe requires aggressive parallelization and strict adherence to integration checkpoints.

Week 1: Infrastructure, Gating, and ASR Baselines (Days 1-7)

The objective of the first week is to establish the continuous audio stream, implement the lightweight gating mechanisms, and validate the transcription backend independently.

- **Days 1-2:** Establish a highly efficient Python multiprocessing loop utilizing sounddevice to capture raw monaural 16 kHz audio.¹¹ Implement a ring buffer that securely holds 30-millisecond chunks and compiles 1-to-2-second rolling arrays.¹²
- **Days 3-4:** Deploy the ONNX INT8 quantized Silero VAD model.¹ Compile and integrate the microwakeword library, configuring the sensitivity thresholds to detect triggers on the local hardware.¹²
- **Days 5-6:** Install the k2-fsa/sherpa-onnx framework and download the SenseVoiceSmall INT8 quantized models.⁸ Validate the Python API using static .wav files to establish baseline inference speeds on the host CPU.²⁸
- **Day 7 (Checkpoint 1):** Demonstrate a functional audio pipeline that passively listens,

detects a wake-word, and routes a single-speaker audio chunk to SenseVoice for accurate transcription.

Week 2: Separation, Biometrics, and Agentic Integration (Days 8-14)

The second week focuses on resolving the computationally intensive center of the pipeline and initializing the reasoning engine.

- **Days 8-9:** Download the pre-trained JusperLee/TIGER-speech safetensors.²⁹ Wrap the inference code into a callable function that accepts the 16 kHz overlapping audio buffer and returns distinct numpy arrays for each separated speaker.³¹
- **Days 10-11:** Deploy the lovemefan/campplus ONNX model.⁷ Implement cosine similarity logic to map the separated audio arrays to predefined team member profiles, generating metadata tags (e.g., ``).¹
- **Days 12-13:** Instantiate the FastMCP framework and configure an SLM (e.g., Qwen2.5-1.5B) via Ollama.¹⁴ Write the foundational Python tools representing smart home actions (e.g., `toggle_lights()`).¹⁴
- **Day 14 (Checkpoint 2):** Connect the separated outputs of TIGER to parallel instances of SenseVoice. Pass the resulting transcribed JSON strings to the MCP agent to trigger a mocked hardware tool.

Week 3: Intent Gating, Polish, and Demo Prep (Days 15-21)

The final week is dedicated to refining the logic tree, implementing the final intent gate to prevent false triggers, and performing rigorous latency optimization.

- **Days 15-16:** Adapt the pre-trained MatchboxNet $3 \times 2 \times 1$ architecture.¹ Integrate it immediately after the wake-word gate to analyze paralinguistic topography and drop ambient conversation buffers.¹
- **Days 17-18:** Program the Python Rule-Based Logic engine to actively evaluate the confidence distributions.¹ Link the Role-Based Access Control (RBAC) definitions to the MCP server to enable hierarchical privilege resolution.¹
- **Days 19-20:** Conduct physical live-testing in a reverberant room. Meticulously monitor memory bandwidth and CPU thermal profiles, tuning ONNX threading parameters if the execution exceeds the 0.5 xRT limit.⁹
- **Day 21 (Checkpoint 3):** Final hackathon presentation recording and live demonstration execution.

Team Task Distribution

The workload must be divided strictly to prevent merge conflicts and ensure both the acoustic processing and the cognitive reasoning systems progress simultaneously.

Engineer Role	Core Responsibilities	Specific Libraries & Models

Engineer A (Acoustics & DSP Lead)	Audio buffering, pipeline gating, ONNX Runtime integration, source separation, and biometric extraction.	sounddevice, numpy, Silero VAD, microwakeword, MatchboxNet, TIGER, CAM++, FAISS.
Engineer B (NLP & Agentic Lead)	ASR sequence transduction, language model prompting, conflict resolution logic, tool building, and terminal UI.	sherpa-onnx, SenseVoiceSmall, FastMCP, Ollama, Qwen2.5-1.5B, SmartSense datasets.

Parallelizable Tasks: While Engineer A tunes the TIGER separation masks using local .wav files, Engineer B can simultaneously build the MCP toolchain and prompt logic using mocked JSON strings representing hypothetical ASR outputs.

Integration Blocker: The primary blocker occurs on Day 14, where the live NumPy arrays generated by Engineer A's TIGER implementation must be successfully passed into Engineer B's Sherpa-ONNX instances without causing memory segmentation faults.

Tooling and Infrastructure

To maximize development speed and guarantee stability during the hackathon, the project relies strictly on battle-tested frameworks tailored for local edge execution.

- **Runtime Environment:** The entire pipeline will execute within a Python 3.10+ virtual environment.
- **Deep Learning Backend:** The Open Neural Network Exchange (ONNX) Runtime serves as the primary inference engine (onnxruntime package).⁹ By converting models from PyTorch to ONNX, the system avoids the massive overhead of loading the full PyTorch library into memory, significantly reducing latency on CPU architectures.¹⁰
- **Audio Processing:** The sounddevice library will manage low-level portaudio streams, while torchaudio and librosa will handle resampling and spectrogram transformations required by the neural networks.⁹
- **Agentic Orchestration:** The FastMCP library provides a lightweight, highly stable implementation of the Model Context Protocol, drastically reducing the boilerplate required to expose Python functions to the LLM.¹⁴
- **Vector Database:** Meta's FAISS (Facebook AI Similarity Search) library will be utilized for the ultra-fast cosine similarity matching of the CAM++ biometric embeddings against the locally enrolled user profiles.¹

Failure Modes and Mitigation Strategies

Deploying a complex, multi-model asynchronous pipeline on consumer hardware introduces severe fragility. Anticipating and mitigating these failure modes is critical for demo survival.

1. Global Interpreter Lock (GIL) Freezing:

- *Risk:* Python's GIL may lock the execution thread when attempting to run two simultaneous instances of the SenseVoice ASR model on the separated audio arrays, causing a massive spike in latency and violating the $xRT < 0.5$ constraint.
- *Mitigation:* Utilize the multiprocessing library rather than standard threading. Alternatively, leverage the inherent C++ multithreading optimizations within the sherpa-onnx framework by carefully managing the `--num-threads` parameter during model initialization.¹¹

2. Acoustic Separation Failure in Extreme Saturation:

- *Risk:* In an exceptionally noisy hackathon hall, the TIGER model may fail to cleanly isolate the harmonic structures, resulting in degraded waveforms that cause the SenseVoice model to hallucinate text.
- *Mitigation:* Rely on the Uncertainty-Propagated Arbitration Engine. If the ASR confidence score drops below 0.50 due to acoustic distortion, the Python logic will bypass the MCP tool execution entirely and trigger a fallback response: "The acoustic environment is too chaotic. Please repeat the command."¹

3. Intent Gating False Positives:

- *Risk:* The MatchboxNet model incorrectly flags a valid command as ambient noise, causing the system to ignore the user.
- *Mitigation:* During the demo, users must be instructed to utilize clear, deliberate paralinguistic tones (a flattened pitch and deliberate pacing) to ensure the network easily distinguishes the command from the chaotic background noise.¹

4. Emergency Simplification Plan:

- If the integration of TIGER proves too computationally intensive on the final days, the emergency fallback is to disable the separation module entirely. The system will revert to a single-stream processor utilizing only Silero, microWakeWord, and SenseVoice. While this abandons the multi-speaker overlap capability, it preserves a highly functional, lightning-fast edge assistant capable of demonstrating agentic tool use and robust wake-word detection.

Final Recommended Stack

The exhaustive evaluation of constraints, parameters, and open-source availability yields the following definitive architectural recommendations.

The "Most Likely to Succeed" Version

This configuration prioritizes absolute stability and implementation speed, ensuring a highly impressive demo within the two-week timeline.

- **VAD:** Silero VAD (ONNX).
- **Trigger:** microWakeWord.
- **Separation:** JuserLee/TIGER-speech (safetensors, 0.8M parameters).
- **Transcription:** sherpa-onnx running SenseVoiceSmall (INT8 quantization).
- **Orchestration:** FastMCP serving tools to Qwen2.5-1.5B-Instruct via Ollama.
- **Implementation Order:** Establish continuous audio buffer -> Integrate ASR -> Build MCP

Agent -> Insert TIGER separation -> Connect pipeline.

The "More Ambitious" Version

If the foundational pipeline is stabilized by Day 12, the team should pursue this advanced configuration to maximize the deep-tech evaluation criteria.

- **Addition 1 (Acoustic Intent Gating):** Integrate the MatchboxNet ($3 \times 2 \times 1$) module immediately after the wake-word to reject false triggers based on paralinguistic analysis, mathematically guaranteeing privacy and compute conservation.¹
- **Addition 2 (Biometric Verification):** Integrate the lovemefan/campplus ONNX model in parallel with the ASR.⁷ Extract vocal embeddings, match them against a local FAISS database, and append explicit user identities to the transcriptions to enable complex, hierarchical Role-Based Access Control within the MCP logic engine.¹

By strictly adhering to this blueprint, the engineering team bypasses the catastrophic pitfalls of joint acoustic modeling, elegantly satisfies the 5.0 million parameter constraint, and successfully orchestrates a highly autonomous, privacy-preserving smart assistant perfectly optimized for dynamic edge environments.

Works cited

1. Architectural Paradigms for Ultra-Lightweight Multi-User Smart Assistants_ Parameter Optimization and Constraint Adherence.pdf
2. Industrial Speech Recognition Toolkit - FunASR, accessed on May 26, 2026, <https://funasr.com/en/>
3. SenseVoice — sherpa 1.3 documentation, accessed on May 26, 2026, <https://k2-fsa.github.io/sherpa/onnx/sense-voice/index.html>
4. MCP Voice Agents - ElevenLabs Showcase, accessed on May 26, 2026, <https://showcase.elevenlabs.io/projects/p/mcp-voice-agents>
5. TIGER: Time-frequency Interleaved Gain Extraction and Reconstruction for Efficient Speech Separation | OpenReview, accessed on May 26, 2026, <https://openreview.net/forum?id=rzx3vcvlzj>
6. JusperLee/TIGER-speech - Hugging Face, accessed on May 26, 2026, <https://huggingface.co/JusperLee/TIGER-speech>
7. lovemefan/campplus: A open-source toolkit for single and multi-modal speaker verification from modelscope and funasr with onnx - GitHub, accessed on May 26, 2026, <https://github.com/lovemefan/campplus>
8. GitHub - k2-fsa/sherpa-onnx: Speech-to-text, text-to-speech, speaker diarization, speech enhancement, source separation, and VAD using next-gen Kaldi with onnxruntime without Internet connection. Support embedded systems, Android, iOS, HarmonyOS, Raspberry Pi, RISC-V, RK NPU, Axera NPU, Ascend NPU, x86_64 servers, websocket server/client, support 12 programming languages, accessed on May 26, 2026, <https://github.com/k2-fsa/sherpa-onnx>
9. ayousanz/cosy-voice3-onnx - Hugging Face, accessed on May 26, 2026, <https://huggingface.co/ayousanz/cosy-voice3-onnx>

10. README.md · ayousanz/cosy-voice3-onnx at 8e103090cf27b29d74a983c2b5283f5210dff563 - Hugging Face, accessed on May 26, 2026,
<https://huggingface.co/ayousanz/cosy-voice3-onnx/blob/8e103090cf27b29d74a983c2b5283f5210dff563/README.md>
11. sherpa-onnx/python-api-examples/simulate-streaming-sense-voice-microphone.py at master - GitHub, accessed on May 26, 2026,
<https://github.com/k2-fsa/sherpa-onnx/blob/master/python-api-examples/simulate-streaming-sense-voice-microphone.py>
12. GitHub - OHF-Voice/micro-wake-word: A TensorFlow based wake word detection training framework using synthetic sample generation suitable for certain microcontrollers., accessed on May 26, 2026,
<https://github.com/OHF-Voice/micro-wake-word>
13. GitHub - modelscope/FunASR: Industrial-grade speech recognition toolkit: 170x realtime, 50+ languages, speaker diarization, emotion detection, streaming, and OpenAI-compatible API., accessed on May 26, 2026,
<https://github.com/modelscope/FunASR>
14. Complete MCP Tutorial: How to Build a Local MCP Voice Assistant in Python - Picovoice, accessed on May 26, 2026,
<https://picovoice.ai/blog/mcp-voice-assistant-tutorial/>
15. GitHub - snudatalab/SmartSense: A dataset repository of "Accurate Action Recommendation for Smart Home via Two-Level Encoders and Commonsense Knowledge" (CIKM 2022), accessed on May 26, 2026,
<https://github.com/snudatalab/smartsense>
16. Speech_Commands.ipynb - Colab - Google, accessed on May 26, 2026,
https://colab.research.google.com/github/tiet-ucs749/tiet-ucs749.github.io/blob/main/matchboxnet/Speech_Commands.ipynb
17. TIGER: Time-frequency Interleaved Gain Extraction and Reconstruction for Efficient Speech Separation - arXiv, accessed on May 26, 2026,
<https://arxiv.org/html/2410.01469v3>
18. CAM++: A Fast and Efficient Network for Speaker Verification Using Context-Aware Masking, accessed on May 26, 2026,
<https://arxiv.org/html/2303.00332v3>
19. GitHub - FunAudioLLM/SenseVoice: Multilingual speech understanding: ASR + emotion recognition + audio event detection. 50+ languages, 15x faster than Whisper, non-autoregressive., accessed on May 26, 2026,
<https://github.com/FunAudioLLM/SenseVoice>
20. funasr/paraformer-zh - Hugging Face, accessed on May 26, 2026,
<https://huggingface.co/funasr/paraformer-zh>
21. Speaker Diarization - LocalAI, accessed on May 26, 2026,
<https://localai.io/features/audio-diarization/index.html>
22. JusperLee/Speech-Separation-Paper-Tutorial - GitHub, accessed on May 26, 2026,
<https://github.com/JusperLee/Speech-Separation-Paper-Tutorial>
23. Recent Trends in Speech Recognition for Diverse Indian Languages, accessed on May 26, 2026,

https://www.researchgate.net/publication/391534125_Recent_Trends_in_Speech_Recognition_for_Diverse_Indian_Languages

24. Accurate Action Recommendation for Smart Home via Two-Level Encoders and Commonsense Knowledge - Hyunsik Jeon, accessed on May 26, 2026, [https://jeon185.github.io/data/CIKM22\(SmartSense\).pdf](https://jeon185.github.io/data/CIKM22(SmartSense).pdf)
25. Kai Li (李凯) JusperLee - GitHub, accessed on May 26, 2026, <https://github.com/JusperLee>
26. domdomegg/microwakeword-python-inference-demo - GitHub, accessed on May 26, 2026, <https://github.com/domdomegg/microwakeword-python-inference-demo>
27. Python API for SenseVoice — sherpa 1.3 documentation - GitHub Pages, accessed on May 26, 2026, <https://k2-fsa.github.io/sherpa/onnx/sense-voice/python-api.html>
28. sherpa/docs/source/onnx/sense-voice/pretrained.rst at master - GitHub, accessed on May 26, 2026, <https://github.com/k2-fsa/sherpa/blob/master/docs/source/onnx/sense-voice/pretrained.rst>
29. JusperLee/TIGER-speech · Discussions - Hugging Face, accessed on May 26, 2026, <https://huggingface.co/JusperLee/TIGER-speech/discussions>
30. TIGER: Time-frequency Interleaved Gain Extraction and Reconstruction for Efficient Speech Separation - GitHub, accessed on May 26, 2026, <https://github.com/JusperLee/TIGER>
31. 3.76 kB - Hugging Face, accessed on May 26, 2026, https://huggingface.co/spaces/WAVbot/TIGER-audio-extraction/resolve/main/inference_speech.py?download=true
32. MCP Voice Assistant - Awesome MCP Servers, accessed on May 26, 2026, <https://mcpservers.org/servers/mcp-use/mcp-use-voice-assistant>